BTS SERVICES INFORMATIQUES AUX ORGANISATIONS **SESSION 2026**

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (recto)

Épreuve E6 - Conception et développement d'applications (option SLAM)

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE	**N° réalisation : RP2**
Nom, prénom : RAFALIMANANTSOA Francky Anthony Marcel	**N° candidat : 02542605405**
Épreuve ponctuelle ☒ **Contrôle en cours de formation** ☐	**Date : 04/03/2026**

Organisation support de la réalisation professionnelle

Le **VMS (Voucher Management System)** est un système de gestion de bons cadeaux développé en une seule modules :

- **Application de bureau JavaFX** (pour les admins, managers, agents)

Intitulé de la réalisation professionnelle

- Développement d'un Voucher Management System, Système de Gestion de Bons Cadeaux.

Période de réalisation : Septembre – Février 2026 **Lieu : MCCI Business School, Ebene**

Modalité : ☐ **Seul(e)** ☒ **En équipe**

Compétences travaillées

☒ Concevoir et développer une solution applicative
☒ Assurer la maintenance corrective ou évolutive d'une solution applicative
☒ Gérer les données

Conditions de réalisation[5] (ressources fournies, résultats attendus)

Ressources fournies :

- Projet en formation.
- Cahier des charges général du projet Voucher Management System

Résultats attendus :

- Application Java/JavaFX fonctionnelle couvrant la gestion complète du cycle de vie des vouchers (création, approbation, génération PDF avec QR code, envoi automatique par e-mail et rédemption en point de vente).
- Base de données PostgreSQL opérationnelle.
- Interface utilisateur ergonomique avec gestion des rôles (Admin, Manager, Agent).

Description des ressources documentaires, matérielles et logicielles utilisées[6]

RESSOURCES DOCUMENTAIRES

Documents fournis par le client :

- Cahier des charges general du projet Voucher Management System

RESSOURCES LOGICIELLES

Serveurs : pgAdmin et Alwaysdata

Environnements de Développement Intégrés (IDE)

IDE professionnels recommandés : IntelliJ IDEA (IDE), Apache Maven 3 (gestion de dépendances et build), PostgreSQL 14+ (SGBD), Git (versioning).

Langages et Technologies

JavaFX 21, iText PDF 5.5.13, ZXing 3.5.1 (QR code), JavaMail 1.6.2, Barcode4J 2.1, SLF4J + Logback (logging). Documentation : documentation officielle Java 21, JavaFX, PostgreSQL JDBC, JavaMail ; tutoriels en ligne ZXing et iText.

Modalités d'accès aux productions[7] et à leur documentation[8]

- Accès à la documentation : https://github.com/0Francky6/VMS-RP2.git
- Accès à la production :

Pour se connecter en tant que :

Administrateur :

- User : admin ;
- Password : admin123

Manager :

- User : manager
- Password : manager123

Approbateur :

- User : approbateur
- Password : approbateur123

---

[5] En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

[6] Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

[7] Conformément au référentiel du BTS SIO « *Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve.* ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

[8] Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS **SESSION 2026**

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)

Épreuve E6 - Conception et développement d'applications (option SLAM)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

1. Contexte et objectifs

Le Voucher Management System (VMS) est une application de bureau développée en Java 21 avec JavaFX 21. Elle permet de gérer l'ensemble du cycle de vie des vouchers (bons de réduction) : saisie de la demande client, validation par workflow (Initiée → Payée → Approuvée / Rejetée), génération automatique du voucher (code unique, QR code, PDF), envoi par e-mail, et rédemption en magasin via scan ou saisie manuelle.

2. Architecture technique

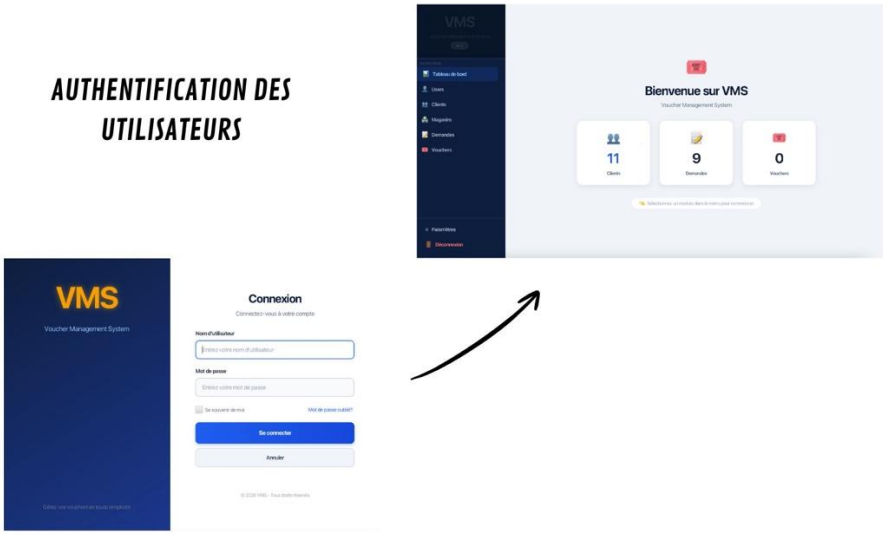
L'application suit le patron MVC (Modèle-Vue-Contrôleur) adapté à JavaFX. Les vues sont définies en FXML et stylisées avec deux feuilles CSS personnalisées (style.css, module.css). Les contrôleurs Java gèrent la logique métier et les accès JDBC à la base de données PostgreSQL. La gestion des dépendances et du build est assurée par Apache Maven 3.

3. Fonctionnalités réalisées

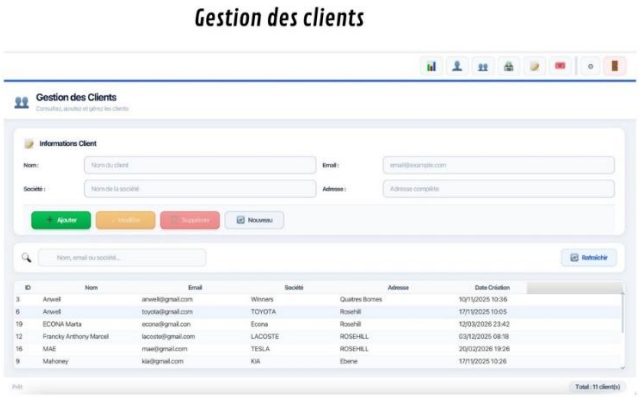
- Gestion de magasins : C'est un module qui gère les **points de vente partenaires**. Il offre un CRUD complet (Créer, Lire, Modifier, Supprimer) sur les magasins.
- Rédemption des vouchers : C'est le module qui permet aux agents en magasin de **valider et enregistrer l'utilisation des bons cadeaux**.

- **Module Login** : authentification par login/mot de passe avec gestion de session et contrôle des droits (Admin, Manager, Agent).

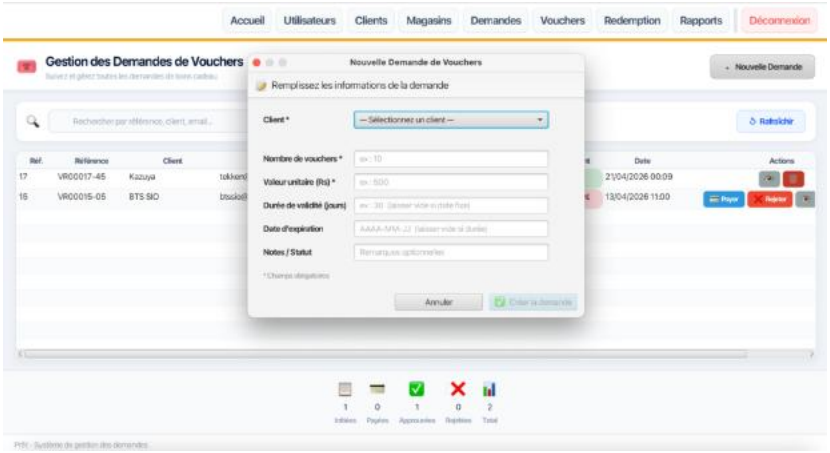
- **Tableau de bord** : statistiques visuelles (demandes par statut, vouchers actifs, rédemptions du jour).



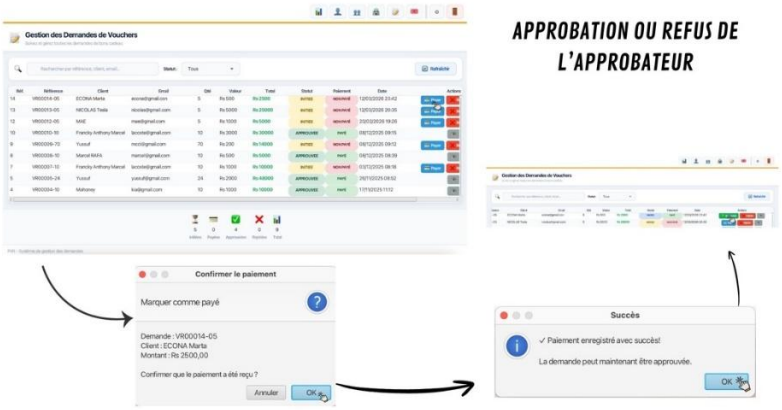
- **Module Clients** : CRUD complet (création, lecture, modification, suppression) avec recherche et filtrage.



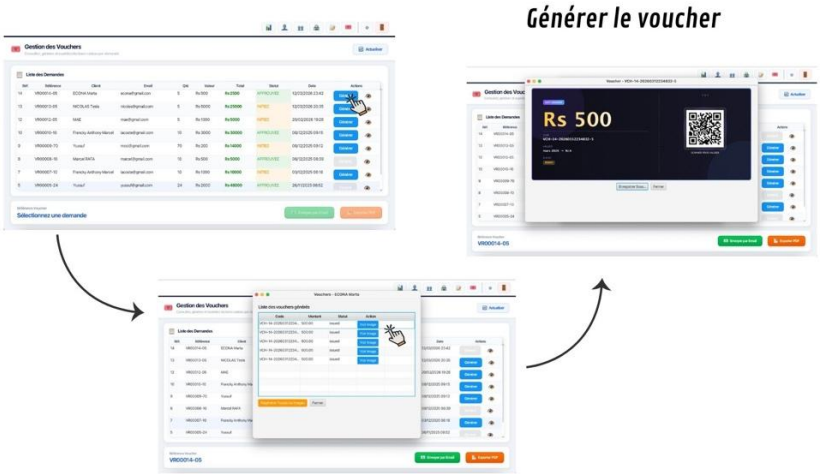
- **Modules Demandes de bon de cadeau** : formulaire de saisie de demandes Client



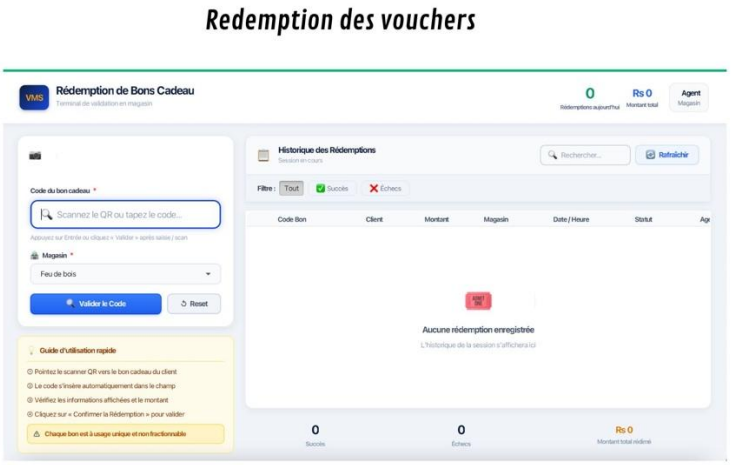
- **Module Demandes** : formulaire multi-étapes avec calcul automatique du montant total, suivi du statut et historique.



- **Module Vouchers** : génération de codes uniques, QR code (ZXing), export PDF (iText), envoi e-mail (JavaMail).

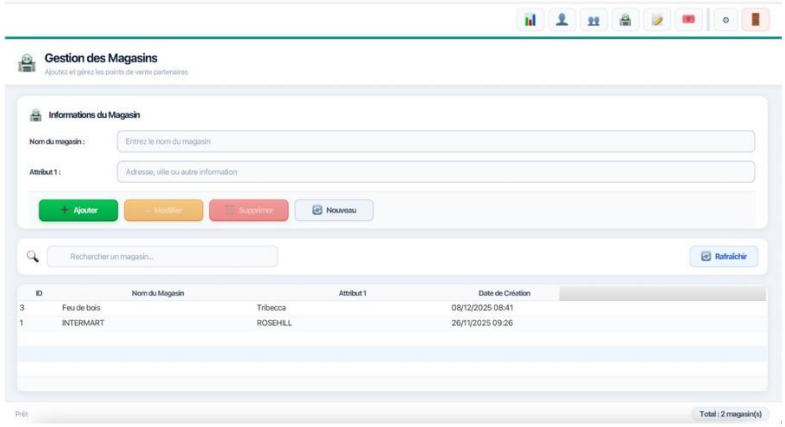


- **Module Rédemption** : scan/saisie du code, validation en temps réel, enregistrement de la rédemption et compteurs de session.



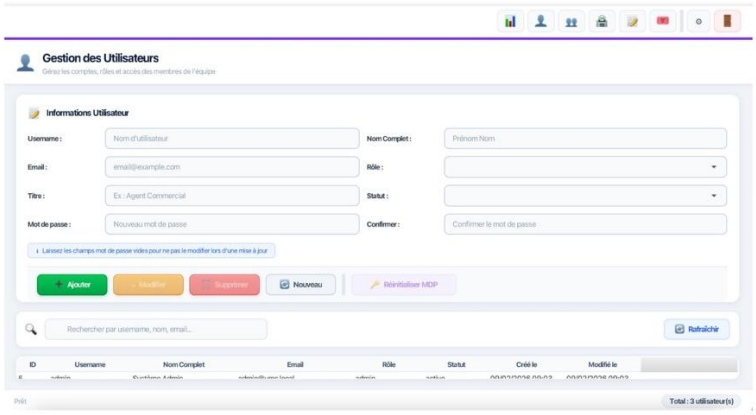
- **Module Magasins** : référentiel des points de vente autorisés.

Gestion de Magasin



- **Module Utilisateurs** : gestion des comptes, rôles et réinitialisation de mots de passe.

Gestion des utilisateurs



4. Productions réalisées

Application JavaFX exécutable (mvn javafx:run). Schéma de base de données PostgreSQL. Feuilles de style CSS. Exemples de vouchers PDF générés (répertoire vouchers/). Rapport de projet détaillé.

5. Compétences mobilisées

Conception et développement d'une solution applicative complète (analyse, modélisation, développement, intégration de bibliothèques tierces). Gestion des données (conception du schéma PostgreSQL, requêtes SQL, JDBC). Maintenance et évolutivité (logging SLF4J, architecture modulaire, CSS séparé du code).

6.Triggers

- **voucher_audit_trigger :** Un trigger qui permet de garder un historique des changements sur les vouchers (insertion et suppression)

CREATE OR REPLACE FUNCTION voucher_audit_fn()

```
RETURNS TRIGGER AS $$
BEGIN
   IF (TG_OP = 'UPDATE') THEN
      IF OLD.status_vouch IS DISTINCT FROM NEW.status_vouch THEN
         INSERT INTO voucher_audit (
            ref_voucher, no_voucher, ancien_statut, nouveau_statut,
            action, details
         ) VALUES (
            NEW.ref_voucher, NEW.no_voucher, OLD.status_vouch, NEW.status_vouch,
            'UPDATE_STATUS',
            'Statut changé de ' || COALESCE(OLD.status_vouch, 'NULL') ||
            ' à ' || COALESCE(NEW.status_vouch, 'NULL')
         );
      END IF;
      RETURN NEW;
   ELSIF (TG_OP = 'INSERT') THEN
      INSERT INTO voucher_audit (
         ref_voucher, no_voucher, ancien_statut, nouveau_statut,
         action, details
      ) VALUES (
         NEW.ref_voucher, NEW.no_voucher, NULL, NEW.status_vouch,
         'INSERT',
         'Nouveau voucher créé avec le statut ' || COALESCE(NEW.status_vouch, 'NULL')
      );
      RETURN NEW;
   ELSIF (TG_OP = 'DELETE') THEN
      INSERT INTO voucher_audit (
         ref_voucher, no_voucher, ancien_statut, nouveau_statut,
         action, details
      ) VALUES (
         OLD.ref_voucher, OLD.no_voucher, OLD.status_vouch, NULL,
         'DELETE',
         'Voucher supprimé avec le statut ' || COALESCE(OLD.status_vouch, 'NULL')
      );
      RETURN OLD;
   END IF;
   RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER voucher_audit_trigger
AFTER INSERT OR UPDATE OR DELETE ON vms.voucher
FOR EACH ROW EXECUTE FUNCTION voucher_audit_fn();
```

- **redem_log_trigger :**

Ce trigger se déclenche lors d'une création de redemption et enregistre un log détaillé dans redemption_logs. Il récupère aussi le numéro du voucher lié pour enrichir les informations de suivi.

```
CREATE OR REPLACE FUNCTION redem_log_trigger_fn()
RETURNS TRIGGER AS $$
DECLARE
   v_no_voucher VARCHAR(100);
BEGIN
```

```
    IF NEW.ref_voucher IS NOT NULL THEN
        SELECT no_voucher INTO v_no_voucher
        FROM vms.voucher
        WHERE ref_voucher = NEW.ref_voucher;
    ELSE
        v_no_voucher := NULL;
    END IF;

    INSERT INTO vms.redemption_logs (
        ref_voucher, no_voucher, magasin_id, redeemed_by,
        statut, motif_echec, log_date, details
    ) VALUES (
        NEW.ref_voucher, v_no_voucher, NEW.magasin_id, NEW.redeemed_by,
        NEW.statut, NEW.motif_echec,
        COALESCE(NEW.date_redemption, CURRENT_TIMESTAMP),
        jsonb_build_object(
            'ref_redemption', NEW.ref_redemption,
            'source', 'trigger:redem_log_trigger'
        )
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER redem_log_trigger
AFTER INSERT ON vms.redemption
FOR EACH ROW EXECUTE FUNCTION redem_log_trigger_fn();
```

- **check_expire_trigger:**

Avant chaque insertion ou mise à jour, il vérifie si un voucher est expiré. Si oui, il change automatiquement son statut en "expired" et met à jour la date de modification.

```
CREATE OR REPLACE FUNCTION check_expire_fn()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status_vouch = 'issued'
       AND NEW.expire_date IS NOT NULL
       AND NEW.expire_date < CURRENT_DATE THEN
        NEW.status_vouch := 'expired';
        NEW.updated_at   := CURRENT_TIMESTAMP;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER check_expire_trigger
BEFORE INSERT OR UPDATE ON vms.voucher
FOR EACH ROW EXECUTE FUNCTION check_expire_fn();
```

7.Procédures stockées

- **expire_vouchers():**

Cette procédure met à jour tous les vouchers expirés en base en leur attribuant le statut "expire". Elle retourne aussi le nombre de vouchers expirés ainsi que leurs identifiants.

```
CREATE OR REPLACE PROCEDURE vms.expire_vouchers(
   OUT vouchers_expires INTEGER,
   OUT vouchers_ids INTEGER[]
)
LANGUAGE plpgsql AS $$
DECLARE
   v_count INTEGER;
   v_ids INTEGER[];
BEGIN
   WITH updated AS (
      UPDATE voucher
      SET status_vouch = 'expire',
         updated_at = CURRENT_TIMESTAMP
      WHERE expire_date < CURRENT_TIMESTAMP
        AND status_vouch NOT IN ('expire', 'redime')
      RETURNING ref_voucher
   )
   SELECT COUNT(*), ARRAY_AGG(ref_voucher)
   INTO v_count, v_ids
   FROM updated;

   vouchers_expires := COALESCE(v_count, 0);
   vouchers_ids     := COALESCE(v_ids, ARRAY[]::INTEGER[]);

   RETURN NEXT;
END;
$$;
```

- **generate_vouchers(p_demande_id) :**

  Cette fonction génère automatiquement plusieurs vouchers à partir d'une demande approuvée. Elle crée les vouchers avec QR code, dates, montants, puis met à jour la demande comme traitée

```
CREATE OR REPLACE FUNCTION vms.generate_vouchers(p_demande_id INTEGER)
RETURNS TABLE (
   ref_voucher  INTEGER,
   no_voucher   VARCHAR(100),
   montant      NUMERIC(12,2),
   status_vouch VARCHAR(50)
)
LANGUAGE plpgsql AS $$
DECLARE
   v_nb_voucher     INTEGER;
   v_valeur_voucher NUMERIC(12,2);
   v_client_id      INTEGER;
   v_duree          INTEGER;
   v_issue_date     TIMESTAMP;
   v_expire_date    TIMESTAMP;
   v_qr_content     TEXT;
   v_voucher_no     VARCHAR(100);
   v_counter        INTEGER;
```

```
    v_timestamp      VARCHAR(20);
BEGIN
    SELECT d.nb_voucher, d.valeur_voucher, d.client_id, COALESCE(d.duree, 365)
    INTO v_nb_voucher, v_valeur_voucher, v_client_id, v_duree
    FROM vms.demande d
    WHERE d.ref_demande = p_demande_id
      AND d.statut = 'approuvee';

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Demande non trouvée ou non approuvée: %', p_demande_id;
    END IF;

    v_issue_date  := CURRENT_TIMESTAMP;
    v_expire_date := v_issue_date + (v_duree || ' days')::INTERVAL;
    v_timestamp   := TO_CHAR(CURRENT_TIMESTAMP, 'YYYYMMDDHH24MISS');

    FOR v_counter IN 1..v_nb_voucher LOOP
        v_voucher_no := 'VCH-' || p_demande_id || '-' || v_timestamp || '-' || v_counter;
        v_qr_content := 'VMS|VOUCHER|' || v_voucher_no;

        INSERT INTO vms.voucher (
            no_voucher, montant, issue_date, expire_date,
            qr_code, status_vouch, client_id, ref_demande
        ) VALUES (
            v_voucher_no, v_valeur_voucher, v_issue_date, v_expire_date,
            v_qr_content, 'issued', v_client_id, p_demande_id
        )
        RETURNING
            voucher.ref_voucher, voucher.no_voucher,
            voucher.montant, voucher.status_vouch
        INTO
            ref_voucher, no_voucher, montant, status_vouch;

        RETURN NEXT;
    END LOOP;

    UPDATE vms.demande
    SET status_demande = 'vouchers_generes'
    WHERE ref_demande = p_demande_id;

    RETURN;
END;
$$;
```

- **log_login_trigger_fn()**

Ce trigger enregistre chaque connexion réussie dans login_logs. Il stocke les infos utilisateur et un message pour garder une trace des accès

```
CREATE OR REPLACE FUNCTION vms.log_login_trigger_fn()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO vms.login_logs (
```

```
        user_id, username, role, statut, message, created_at
    ) VALUES (
        NEW.user_id, NEW.username, NEW.role,
        'succès',
        'Connexion réussie pour ' || NEW.username,
        NOW()
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER log_login_trigger
AFTER INSERT ON vms.users_connected
FOR EACH ROW EXECUTE FUNCTION vms.log_login_trigger_fn();
```